ETH Zürich, 227-0421-00

Learning in Deep Artificial and Biological Neuronal Networks

Instructors: Prof. Dr. Benjamin F. Grewe

Dr. Pau Vilimelis Aceituno

ETH zürich

Joonsu Gha, Michele Viscione

HS2026

Contents

1 Prerequisites **3**
1.1 Dot product: . . . 3
1.2 Outer product: . . . 3
1.3 Jacobian: . . . 3
1.4 Chain Rule: . . . 3

2 Back-propagation: A geometric approach **4**
2.1 Optimizing linear relations . . . 4
2.2 Bias and batches . . . 5
2.3 Finally Deep Nets . . . 6
2.4 Example . . . 8

3 Feedback Alignment: A Biologically Plausible Alternative **10**
3.1 The Core Idea of Feedback Alignment . . . 10
3.2 Conditions on B_k . . . 10
3.3 Biological Plausibility . . . 11
3.4 Direct Feedback Alignment (DFA) . . . 12

1 Prerequisites

1.1 Dot product:

Given $a, b \in \mathbb{R}^n$, $a^T b \in \mathbb{R}$ where

$$a^T b = \sum_i a_i b_i$$

Notation: $\langle a, b \rangle$, $a \cdot b$, $a^T b$
Observe that $a^T b = \|a\|\|b\| \cos \theta_{ab}$

1.2 Outer product:

Given $a \in \mathbb{R}^m$ $b \in \mathbb{R}^n$, $ab^T = M \in \mathbb{R}^{m \times n}$, where

$$M_{ij} = a_i b_j$$

1.3 Jacobian:

Given $x \in \mathbb{R}^n$ and $f : \mathbb{R}^n \to \mathbb{R}^m$

$$\frac{\partial f}{\partial x} = J_x f = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

1.4 Chain Rule:

Given $x \in \mathbb{R}^n$ and $h : \mathbb{R}^n \to \mathbb{R}^m$, $g : \mathbb{R}^n \to \mathbb{R}^d$, $f : \mathbb{R}^d \to \mathbb{R}^m$

$$h(x) = f(g(x)) \implies h'(x) = f'(g(x))g'(x)$$

If we use the convention for which the gradients are column vectors ($\nabla_x f = \left(\frac{\partial f}{\partial x}\right)^T$) the multivariable chain rule becomes

$$\nabla_x h(x) = \left(\frac{\partial h}{\partial x}\right)^T = \left(\frac{\partial g}{\partial x}\right)^T \nabla_{g(x)} f = J_x g^T \nabla_{g(x)} f$$

2 Back-propagation: A geometric approach

2.1 Optimizing linear relations

Given an input $x \in \mathbb{R}^n$, a target $y \in \mathbb{R}^m$, we want to update our "weights" matrix $W \in \mathbb{R}^{m \times n}$ to make the inference $\hat{y} = Wx$ closer to y.

Claim: Given a loss function $\mathcal{L}(\hat{y}, y)$, the following weight update ΔW optimizes $\mathcal{L}$ through gradient descent:

$$\Delta W = (\nabla_{\hat{y}} \mathcal{L}) x^T$$

With $W^* = W - \eta \Delta W$, $\eta \in \mathbb{R}^+$.

Let's check what happens if we compute $W^* x$:

$$W^* x = (W - \eta \Delta W)x = \hat{y} - \eta (\nabla_{\hat{y}} \mathcal{L}) x^T x = \hat{y} - \eta \|x\|^2 \nabla_{\hat{y}} \mathcal{L}$$

So this weight update is equivalent to performing a gradient descent step *in the output space*, with step size $\eta \|x\|^2$.

NB: as an example, the loss $\mathcal{L} = \frac{1}{2}\|\hat{y} - y\|^2 = \frac{1}{2}(\hat{y} - y)^T(\hat{y} - y)$ leads to $\nabla_{\hat{y}} \mathcal{L} = (\hat{y} - y)$. Therefore, naming $\alpha = \eta ||x||^2$, we get

$$W^* x = \hat{y} - \eta \|x\|^2 (\hat{y} - y) = \alpha y + (1 - \alpha)\hat{y}$$

Observe that the gradient descent step works as an interpolation between the current activity $\hat{y}$ and the target activity y. In order for this step to be stable, we need $\alpha \leq 1$, and so $\eta \leq \frac{1}{\|x\|^2}$.

Intuition: the outer products ab^T can be interpreted as a made by two specific input and output functions. Let's assume I want to apply ab^T as a transformation to some vector v, this can be broke down in the following steps:

1. Compute alignment $b^T v = \|b\| \|v\| \cos \theta_{bv}$
2. The final output is $a \|b\| \|v\| \cos \theta_{bv}$

So, the more my input aligns to b, the more the output is proportional to a. Likewise, the outer product defined before informally says "next time you see something that reminds you of x, push it a bit more toward y, proportionally to the gradient $\nabla_{\hat{y}} \mathcal{L}$".

2.2 Bias and batches

Shorthand notation for average:

Given $\{b_i\}_{i=1,\dots N}$, $b_i \in \mathbb{R}^n$, we have $\bar{\sum}_i b_i = \frac{1}{N}\sum_i b_i$

Now we are given a set of inputs an targets $\{x^j, y^j\}_j$, a *batch*, with $x^j \in \mathbb{R}^n$ and $y^j \in \mathbb{R}^m$. This time we want to update not only $W \in \mathbb{R}^{m\times n}$ but also a bias vector $b \in \mathbb{R}^m$. Our inference becomes $\hat{y}^j = Wx^j + b$. The update for W is simply the average of the batch-element-specific updates, which are the same as the one mentioned before:

$$\Delta W = \bar{\sum_j} \nabla_{\hat{y}^j} \mathcal{L}(\hat{y}^j, y^j) {x^j}^T$$

For the bias, we will update it using the average of the output gradients:

$$\Delta b = \bar{\sum_j} \nabla_{\hat{y}^j} \mathcal{L}(\hat{y}^j, y^j)$$

In a standard gradient descent setting, we can apply the following update rule:

$$W^* = W - \eta \Delta W$$
$$b^* = b - \eta \Delta b$$

Intuition: we want b to capture the average output activity (this can be helpful to leverage downstream non-linearities). Therefore, we update it using the mean of updates across the batch!

As before we can check what's the impact in the output space:

$$W^* x^i + b^* = \hat{y}^i - \eta \bar{\sum_j} (\langle x^j, x^i \rangle + 1) \nabla_{\hat{y}^j} \mathcal{L}(\hat{y}^j, y^j)$$

NB: Usually more sophisticated updates rules $W^* = O(W, \Delta W, \dots)$ are possible, in Deep Learning this refers to the choice of an *optimizer* (SGD, momentum based, Adam, ...).

2.3 Finally Deep Nets

We now have all the ingredients we need to backpropagate on arbitrary function chains. Let's consider a L-layered Neural Network, where the k-th layer is given by the weights $W_k \in \mathbb{R}^{d_{k+1} \times d_k}$, a bias $b_k \in \mathbb{R}^{d_{k+1}}$ and activation function ϕ_k. The intermediate output of our network at the k-th layer is given by

$$x_{k+1} = \phi_k(z_k) = \phi_k(W_k x_k + b_k) = \phi_k(W_k \phi_{k-1}(z_{k-1}) + b_k) = \dots$$

Therefore we have

- The NN's ouput is $\hat{y} = x_{L+1}$
- The NN's input is $x = x_1$
- The pre-activations vectors are z_k
- The post-activation vectors are x_k

How do we compute ΔW_i, Δb_i for the i-th layer? Same as before! We use the outer product between the loss gradient with respect to the pre-activation output (in this case z_i) and the layer input x_i.

$$\Delta W_i = \underbrace{\nabla_{z_i} \mathcal{L}}_{\text{output's gradient}} \underbrace{x_i^T}_{\text{input}}$$

Let's unpack $\nabla_{z_i} \mathcal{L}$ using the multi-variable chain rule:

$$\nabla_{z_i} \mathcal{L} = \left(\frac{\partial \hat{y}}{\partial z_i}\right)^T \nabla_{\hat{y}} \mathcal{L} = \left(\frac{\partial \hat{y}}{\partial z_L} \frac{\partial z_L}{\partial x_L} \frac{\partial x_L}{\partial z_{L-1}} \cdots \frac{\partial x_{i+1}}{\partial z_i}\right)^T \nabla_{\hat{y}} \mathcal{L}$$

If we name

$$J_{\phi_k} = \frac{\partial x_{k+1}}{\partial z_k}$$

$$J_{z_k} = \frac{\partial z_k}{\partial x_k} = W_k$$

We can rewrite $\nabla_{z_i} \mathcal{L}$ as:

$$\nabla_{z_i} \mathcal{L} = \left(J_{\phi_L} J_{z_L} J_{\phi_{L-1}} \cdots J_{z_{i+1}} J_{\phi_i}\right)^T \nabla \mathcal{L}_{\hat{y}} = \left(\prod_{k=i}^{L-1} J_{\phi_k}^T W_{k+1}^T\right) J_{\phi_L}^T \nabla_{\hat{y}} \mathcal{L}$$

Jacobian of an activation function:

Activation functions are usually element-wise, therefore, their Jacobian is a diagonal matrix (no interaction between different entries of the vector). One of the most used is the ReLU activation function:

$$\phi(z) = ReLU(z) = \max(0, z)$$

its Jacobian is a diagonal matrix, where the k-th diagonal element is 1 when the k-th entry of z is > 0 and 0 otherwise.

$$J_\phi^j = D_{>0}(z^j) = \text{diag}(1 \text{ if } (z^j)_k > 0, \text{else } 0)$$

Similarly, other element-wise activation functions like sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ results in a diagonal Jacobian $J_\sigma^j = \text{diag}(\sigma'((z^j)_k))$. Depending on the task, the final activation function ϕ_L can be absent; in that case we consider it as the identity function and its Jacobian becomes the identity matrix.

Finally, for a batch $\{x^j, y^j\}_j$, the update for the i-th weights and bias is

$$\Delta W_i = \bar{\sum_j} \nabla_{z_i^j} \mathcal{L}(\hat{y}^j, y^j) {x_i^j}^T = \bar{\sum_j} \Big(\prod_{k=i}^{L-1} {J^j}_{\phi_k}^T W_{k+1}^T \Big) {J^j}_{\phi_L}^T \nabla_{\hat{y}} \mathcal{L}(\hat{y}^j, y^j) {x_i^j}^T$$

$$\Delta b_i = \bar{\sum_j} \nabla_{z_i^j} \mathcal{L}(\hat{y}^j, y^j) = \bar{\sum_j} \Big(\prod_{k=i}^{L-1} {J^j}_{\phi_k}^T W_{k+1}^T \Big) {J^j}_{\phi_L}^T \nabla_{\hat{y}} \mathcal{L}(\hat{y}^j, y^j)$$

I know it's ugly, but the foundational meaning remains the same as in the linear case. The difference is that now it is harder to compute the gradient of a layer's output, as it depends on all its subsequent layers. We will soon try to tame this with an example. Before that, from this equation it became clear that the different layers must be updated *sequentially*, top to bottom, as the update layer i requires information from the next layers. The term *backpropagation* originates exactly from this process: after each layer update, we need to propagate some information backward to update the previous ones. We will now make this algorithm explicit.

Why "propagation"?

We can build the backpropagation algorithm iteratively by exploiting the following relation

$$\nabla_{z_k} \mathcal{L} = J_{\phi_k}^T W_{k+1}^T \nabla_{z_{k+1}} \mathcal{L}$$

with $\nabla_{z_k} \mathcal{L}$ often referred in literature as δ_k. Therefore at each layer, with batch index s, we perform the following steps:

1. $\delta_k^s \leftarrow {J_{\phi_k}^s}^T W_{k+1}^T \delta_{k+1}^s$

2. $\Delta b_k \leftarrow \bar{\sum}_s \delta_k^s$
3. $\Delta W_k \leftarrow \bar{\sum}_s \delta_k^s x_k^{s\,T}$
4. "propagate" δ_k to layer $k-1$

This holds true for $k \in [1, L-1]$, while if $k = L$ we simply have:

$$\delta_L^s \leftarrow J_{\phi_L}^{s} \nabla_{\hat{y}^s} \mathcal{L}$$

For a single layer, the corresponding weight update is known as the "delta rule".

2.4 Example

Let us consider a 2-layer NN, with a ReLU function for ϕ_1 and no ϕ_2 (identity). The NN's output is given by:

$$\hat{y} = W_2 \phi_1(W_1 x + b_1) + b_2$$

With $x_2 = \phi_1(z_1 + b_1) = \phi_1(W_1 x_1 + b_1)$. We study the layer updates for a batch $\{x^s, y^s\}_s$ and loss function $\mathcal{L} = \frac{1}{2}\|\hat{y}^s - y^s\|^2$.

Layer-2:

We know from before that

$$\Delta W_2 = \bar{\sum_s} \nabla_{\hat{y}^s} \mathcal{L}(\hat{y}^s, y^s) x_2^{s\,T}$$

For a given sample, we have

$$\nabla_{\hat{y}^s} \mathcal{L}(\hat{y}^s, y^s) = (\hat{y}^s - y^s)$$

Therefore

$$\Delta W_2 = \bar{\sum_s} (\hat{y}^s - y^s) x_2^{s\,T}$$

$$\Delta b_2 = \bar{\sum_s} (\hat{y}^s - y^s)$$

Layer-1:

Similarly, for the update of the first layer we need to compute

$$\nabla_{z_1^s} \mathcal{L} = J_{\phi_1}^{s\,T} W_2^T \nabla_{\hat{y}^s} \mathcal{L}(\hat{y}^s, y^s) = D_{>0}(z_1^s) W_2^T (\hat{y}^s - y^s)$$

Finally, we obtain

$$\Delta W_1 = \bar{\sum_s} D_{>0}(z_1^s) W_2^T (\hat{y}^s - y^s) x^{sT}$$

$$\Delta b_1 = \bar{\sum_s} D_{>0}(z_1^s) W_2^T (\hat{y}^s - y^s)$$

3 Feedback Alignment: A Biologically Plausible Alternative

Feedback alignment (FA) [Lillicrap et al. (2014)], is an alternative to the standard backpropagation algorithm used in training neural networks. As we saw in the previous sections, in traditional backpropagation gradients are computed by chain rule and propagated backward through the network using the transposed weight matrices of each subsequent layer. This means that the error signal $\delta_k = \nabla_{z_k} \mathcal{L}$ at layer k is computed using the weights W_{k+1} of the next layer:

$$\delta_k \leftarrow {J_{\phi_k}}^T W_{k+1}^T \delta_{k+1}$$

This dependence on the transpose of the forward weights presents several challenges, particularly from a biological plausibility perspective.

3.1 The Core Idea of Feedback Alignment

Feedback alignment proposes a more biologically plausible learning rule by using fixed random feedback weights, rather than transposed forward weights (W_k^T), to propagate errors back through the network. Instead of W_k^T, each layer k is associated with a feedback matrix B_k. Thus, the error signal at layer k is calculated as:

$$\delta_k \leftarrow {J_{\phi_k}}^T B_{k+1} \delta_{k+1}$$

Here, similiraly to the BP case, δ_{k+1} represents the error signal backpropagated from layer $k+1$ (or the output loss gradient if $k = L$). The matrices B_k are randomly initialized (e.g., with elements drawn from a standard normal distribution) and remain fixed throughout the training process. Despite using these "incorrect" feedback paths, FA has been shown to train deep neural networks effectively, achieving performance comparable to backpropagation on various tasks.

3.2 Conditions on B_k

The effectiveness of feedback alignment hinges on the properties of these random feedback matrices B_k. The key insight from the Lilicrap paper is that for FA to work, the feedback weights B_k do not need to be the transpose of the forward weights W_k. Instead, the crucial requirement is that the feedback pathways B_k are "aligned" with the true gradient direction *on average*.

While B_k is random and fixed, the network learns to adjust its forward weights W_k such that the effective gradient provided by B_k becomes useful for learning. Specifically, Lilicrap et al. show that if the forward weights W_k are sufficiently large, the alignment between B_k and W_k^T improves during training. This phenomenon is termed "alignment," where the direction of the random feedback gradient eventually aligns with the true backpropagation gradient.

The initial random values of B_k are typically drawn from a distribution with zero mean and appropriate variance (e.g., standard normal distribution or a scaled uniform distribution) to ensure numerical stability and proper signal propagation.

3.3 Biological Plausibility

One of the primary motivations for Feedback Alignment is its enhanced biological plausibility compared to traditional backpropagation. The challenges for biological implementation of BP include:

1. **Weight Transposition Problem:** In standard backpropagation, the feedback connections must use the transpose of the forward weights (W_k^T). This implies that for every synapse W_{ij} connecting neuron i to neuron j in the forward pass, there must be a corresponding feedback connection from neuron j to neuron i with a weight W_{ji} such that $W_{ji} = W_{ij}$. This "transpose symmetry" is difficult to reconcile with known neural architectures and plasticity mechanisms. It would require neurons to dynamically "mirror" their synaptic strengths for both forward and backward error propagation.
2. **Symmetry Problem (Weight Sharing):** More generally, the update for layer k in BP requires information about the weights W_{k+1} of the subsequent layer. This implies a "communication" or "coordination" mechanism across layers that is not immediately apparent in biological neural networks. A neuron cannot easily access the precise synaptic weights of neurons in downstream layers to compute its error signal.
3. **Timing and Non-locality:** Backpropagation requires a sequential backward pass, implying specific timing and information flow that is complex to implement in a highly parallel and asynchronous biological system. Each layer needs the error signal from the layer above it, necessitating a global coordination of information flow that is hard to achieve with local learning rules.

Feedback alignment circumvents most of these issues by decoupling the forward and backward pathways. Since the feedback matrices B_k are random and fixed, there is no need for weight transposition or complex coordination between forward and backward weights. The error signal for a layer is derived solely from the output error and the fixed feedback path, making the learning rule more local and less demanding in terms of information transfer, resembling more closely the local synaptic plasticity mechanisms observed in the brain. However, the non-locality issue has only been partially solved. Although the feedback path is fixed, sequential computation is still required, which contrasts with the more parallel architectures present in biology. To further strengthen this intuition, we will now examine a non-sequential variant of

FA with direct feedback paths.

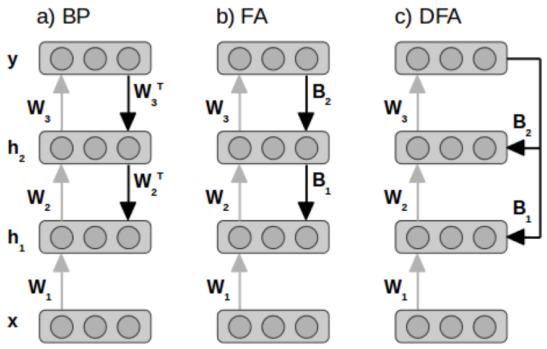


Figure 3.1: Overview of different error transporation configurations. (a) Backpropagation (b) Feedback Alignment (c) Direct Feedback Alignment. Source: Nøkland (2016)

3.4 Direct Feedback Alignment (DFA)

A further simplification and extension of Feedback Alignment is Direct Feedback Alignment (DFA) [Nøkland (2016)]. In standard FA, error signals propagate layer by layer (e.g., δ_{k-1} depends on δ_k). In DFA, the error signal for every hidden layer k is directly calculated from the overall network output error $\delta_L = \nabla_{\hat{y}}\mathcal{L}$, using a dedicated random feedback matrix B_k that maps the output error directly to that hidden layer:

$$\delta_k \leftarrow {J_{\phi_k}}^T B_k \nabla_{\hat{y}}\mathcal{L}$$

Here, B_k is a random matrix specific to layer k that directly connects the output error to the pre-activation of layer k, as shown in Figure 3.1 (c). This significantly simplifies the error propagation path, removing the dependency on intermediate error signals. DFA further enhances biological plausibility by localizing the error signal, allowing each hidden layer to receive a direct global error signal without relying on information from adjacent layers in the backward pass. This is conceptually similar to a "teacher" signal directly informing each part of the network about the overall performance error.

Both FA and DFA demonstrate that exact gradient computation might not be strictly necessary for effective learning in deep networks, opening avenues for alternative learning paradigms that are more aligned with the constraints of biological computation.

Bibliography

Lillicrap, T. P., D. Cownden, D. B. Tweed, and C. J. Akerman (2014). Random feedback weights support learning in deep neural networks. *arXiv preprint arXiv:1411.0247*.

Nøkland, A. (2016). Direct feedback alignment provides learning in deep neural networks. *Advances in neural information processing systems 29*.